

행동 기반 보안 캡차 원리와 방어 구조

오브젝트 드래그 + 행동분석 CAPTCHA

"무엇을 골랐는가"가 아니라 "어떻게 움직였는가"를 검증하는 캡차의 작동 원리, 13겹 방어 스택, 행동 AI, 그리고 개발 과정에서 규명한 핵심 기술 발견을 정리한다.

1 개요 — 왜 행동을 검증하나

2 작동 원리 — 4단계

3 13겹 방어 스택

4 위조 방지의 핵심

5 행동 AI 파이프라인

6 핵심 기술 발견 (연구 노트)

7 시스템 아키텍처 · 현황

2026.08 · CONFIDENTIAL

1 개요 — 왜 행동을 검증하나

기존 캡차의 전제가 무너졌기 때문이다.

문제

문자 왜곡·이미지 선택은 "AI가 사람보다 못 본다"는 전제 위에 있었다. 오늘날 OCR·비전 모델은 사람보다 정확히 읽는다. '인식을 시험하는' 캡차는 이미 AI에게 유리한 시험이다. 진짜 위협은 조잡한 스크립트가 아니라 브라우저 자동화(Playwright 등)로 사람과 유사한 타이밍·궤적을 만드는 정밀 봇이다.

접근

CatChap은 인식이 아니라 **행동**을 검증한다. 사용자는 지정 사물을 정답 영역으로 **드래그**하면 된다(약 3초). 그 사이 발생하는 포인터 궤적·속도·원천 신호를 서버가 **위조 불가능하게** 수집·분석해 사람다움을 판별한다. 정답을 알아도 사람처럼 움직이지 않으면 통과하지 못한다.

핵심 질문의 이동: "무엇을 골랐는가" → "어떻게 움직였는가". 여기에 PoW·허니팟·문항회전·자동화탐지를 겹쳐 단일 우회로가 없는 다층 방어를 구성한다.

2 작동 원리 — 4단계 파이프라인

모든 단계에 방어가 내장되어, 각 단계가 다음 단계의 신뢰 근거가 된다.

① 챌린지 생성 Server

문항풀에서 최소노출·쿨다운 기준으로 문항을 고르고 방어 장치를 심어 발급한다. 은닉 히트영역(정답 위치를 투명 처리)·임시 ID 리매핑(매번 새 값→리플레이 차단)·허니팟(투명 함정)·드롭존 랜덤화·적응형 PoW 시드.

② 클라이언트 풀이 Browser

사용자는 드래그만 하지만, 위젯은 뒤에서 **SHA-256 PoW**를 백그라운드 워커로 연산하고, 포인터 이벤트를 **배치채널(영수증 체인)**로 서버에 실시간 스트리밍한다. 각 조각은 이전 조각의 영수증에 묶여 중간 조작을 감지한다.

③ 다단 검증 Server

여러 관문을 순서대로 통과해야 한다 — PoW 게이트 → 허니팟 즉시차단 → 정답 판정 → 행동 휴리스틱 채점 → 행동 AI 채점 → 자동화·클러스터 탐지. 하나라도 걸리면 차단 또는 재도전(step-up).

④ 토큰 발급·재검증 Server ↔ Site

통과 시 **일회성 토큰**을 발급하고, 도입 사이트가 사이트 시크릿으로 **서버-대-서버 재검증**한다. 재사용 불가.

3 13겹 방어 스택 · 5계층

성격이 다른 13개 방어를 5계층으로 겹쳐 우회 비용을 비대칭적으로 끌어올린다.

L1 입구	① 적응형 Proof-of-Work	세션 위협도로 난이도 상향 · 연산비용
L2 문제	② 은닉 히트영역	정답 위치 은닉
	③ 임시 ID 리매핑	매 요청 새 값 · 리플레이 차단
	④ 허니팟 함정	접촉 즉시 봇 확정
	⑤ 드롭존 랜덤화	좌표 하드코딩 무력화
L3 행동	⑥ 신뢰 이벤트 배치채널(영수증 체인)	서버측 위조불가 원본 확보
	⑦ 행동 휴리스틱 채점	속도·궤적·타이밍 규칙 판별
	⑧ behavior-AI (LightGBM)	행동 학습 모델 채점(shadow)
L4 정황	⑨ 자동화 · 헤드리스 탐지	webdriver·headless 신호 접수화
	⑩ 클러스터 탐지 (행동 지문)	동일 패턴 대량 세션 군집
L5 경계	⑪ 레이트리밋	IP당 요청 제한
	⑫ Origin · CSP 검증	도메인 잠금 · 프레임링 차단
	⑬ 문항 회전	최소노출 · 쿨다운 노출 분산

적응형 PoW

정상 사용자에게 수백 ms의 무해한 연산, 대량 자동화엔 세션마다 누적되는 비용의 벽. 위협 세션엔 난이도 자동 상향(17→21비트).

허니팟

사람 눈엔 안 보이는 투명 함정을 실객체와 구분 불가하게 섞어 배치. 접촉 시 봇 확정. (채점·기록 후 차단해 봇 궤적을 라벨로 확보)

문항 회전

최소노출 + 30분 쿨다운으로 같은 문항 반복 노출 방지. 문항 유출·암기 자동화를 늦춘다.

배치채널

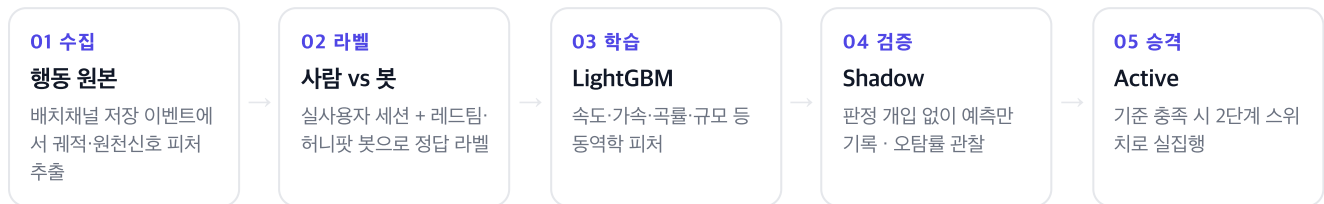
행동을 조각조각 서버에 저장하고 영수증으로 체인. 최종 판정은 클라이언트 제출값이 아니라 이 서버 원본으로 한다.

4 위조 방지의 핵심

최종 판정은 클라이언트가 제출한 값이 아니라, 배치채널로 서버에 미리 저장된 행동 원본으로 이뤄진다. 봇이 verify 요청에 "사람처럼 보이는 가짜 궤적"을 실어도 소용없다 — 서버는 이미 저장된 원본을 보고, 각 배치가 영수증 체인으로 묶여 중간 조작·누락·재전송을 감지한다. 좌표 해시는 6자리 정규화로 저장-재검증 간 정밀도 드리프트까지 막는다.

5 행동 AI 파이프라인

"어떻게 움직였는가"를 학습하는 모델. 안전하게 검증(shadow)한 뒤에만 실집행으로 승격한다.



모델 — LightGBM

그래디언트 부스팅 결정트리. 속도평균·가속·궤적 곡률·이벤트 수·총시간·정지-재개 패턴 등 **정형(tabular) 피쳐**로 사람/봇을 가른다. 적은 데이터로도 잘 학습되고, 실시간 추론이 빠르며, 특징 중요도로 "무엇을 보는지" 해석 가능해 행동 판별에 적합하다. 별도 마이크로서비스로 분리 운영하며 **SHADOW** 상태(판정 미개입).

수집하는 신호

분류	신호	의미
궤적	x·y·timestamp	이동 경로·속도·가속·곡률·간격 CV
PointerEvent 원천신호	is_trusted pointer_type · pressure · buttons · coalesced_count	JS 합성 이벤트면 false — 스크립트 봇 직격 입력장치 특성 · 프레임당 원시 샘플 수(실브라우저 없이 생성 불가)

안전한 2단계 승격

모델이 사람을 봇으로 오판(false positive)하면 정상 사용자가 막힌다. 그래서 먼저 shadow로 충분한 표본에서 **오탐률·참여자별 통과율**을 확인하고, go/no-go를 통과한 뒤에만 (캡차 정책 active + 모델 candidate→active) 실집행한다. 승격 전까지는 PoW·허니팟·휴리스틱이 안전망이다.

6 핵심 기술 발견 (연구 노트)

레드팀 봇을 직접 만들어 통제 실험으로 규명한 것들.

① 판별 측은 "궤적 모양"이 아니라 "상호작용 규모"

궤적·타이밍·도구를 모두 고정하고 **드래그 개수만** 바꿨더니 모델 점수가 뒤집혔다.

조건 (직선 궤적 · 실시간 타이밍 고정)	사람일 확률	판정
drag=all — 여러 객체 끌기(~70 이벤트)	0.9987	사람 5/5
drag=one — 1개만 끌기(~14 이벤트)	0.10	봇 9/10

함의(방어 관점 주의): 자로 그은 직선이어도 **여러 번 끌면 사람으로 통과**한다. 봇에게 "여러 번 끌기"는 거의 공짜인 반면 궤적 위조는 어렵다 → 쉬운 쪽으로 뚫린다. 이 발견으로 "규칙적 간격=봇", "속도/총시간이 축" 가설이 모두 기각됐다. (미분리: 규모=이벤트수·총시간·드래그횟수가 동반 이동 → 통제 sweep으로 분해 예정)

② 궤적 밖 신호가 필요하다 — PointerEvent 원천신호

(x,y,시각)은 공격자가 마음대로 만드는 값이라 거기서만 뽐은 특징은 더 좋은 생성기에 따라잡힌다. 그래서

`is_trusted` · `coalesced_count` 등 **실브라우저 없이는 못 만드는 신호** 9필드를 추가 수집한다. 단, 현재 신호는 "브라우저를 썼는가"를 가릴 뿐 — Playwright 봇은 `is_trusted=true` 라 사람과 구분되지 않는다. 실질 후보는 `coalesced_count` (프레임당 원시 move 수)이며, 사람 마우스(고폴링)가 트랙패드·봇보다 큰 값을 내는지가 관건이다.

③ secure-context 제약 — HTTP vs HTTPS

`crypto.randomUUID` · `getCoalescedEvents` · `crypto.subtle` 은 **secure-context(HTTPS/localhost) 전용**이다. 평문 HTTP에선 `randomUUID`가 없어 캡차가 아예 안 뜨던 문제를 `getRandomValues` 폴백으로 해결했고, `coalesced` 측정은 HTTPS가 필요하다(프로덕션은 HTTPS라 존재 — 수집 환경의 문제).

④ 방어층 점검 — PoW 워커

번들 미니파이 시 워커 본문의 함수명이 어긋나 워커가 throw하던 것을 대입 방식으로 고쳤다(메인스레드 폴백은 동작해 PoW 자체는 유지됐고, 서버는 PoW 없는 verify를 실측 기준 거부함을 확인).

7 시스템 아키텍처 · 현황

관심사를 분리한 모듈형 구조. 캡차 로직과 행동 모델을 독립 서비스로 나눠 안전하게 배포·롤백한다.

① 임베드 위젯 Frontend

React 경량 위젯, 드래그 UI · PoW Web Worker · 행동 스트리밍.

② 캡차 API Core

FastAPI, 챌린지·검증·토큰·관리 API, 방어 스택 대부분 집행.

③ 데이터 스토어 Data

MySQL, 문항·챌린지·토큰·행동 원본·모델 예측, 운영/학습 역할 분리.

④ 행동 모델 서비스 AI

LightGBM 내부 전용 마이크로서비스, 외부 미노출, Shadow/Active.

스택

Frontend	React · Web Worker · Vite
Backend	Python · FastAPI
Data	MySQL
AI	LightGBM (별도 마이크로서비스, shadow)
Ops	systemd · 워크트리 빌드→rsync→무중단 재시작 · 즉시 롤백

현황

- LIVE** 오브젝트 드래그 캡차 · 적응형 PoW · 허니팟 · 드롭존 랜덤화 · 문항 회전
- LIVE** 신뢰 이벤트 배치채널 · 행동 휴리스틱 · 자동화/클러스터 탐지 · 레이트리밋 · Origin/CSP
- LIVE** PointerEvent 원천신호 9필드 수집
- SHADOW** 행동 AI(LightGBM) 예측 상시 기록
- NEXT** 실트래픽·장치 다양성 수집 → 오탐률 검토 → active 승격 / 정밀봇 후속 방어

검수 완료 문항풀 **1,660여 개** 운영 · 방어 기법 **13겹(5계층)**. 봇 차단은 완결이 아니라 군비경쟁이며, CatChap의 강점은 절대적 무결이 아니라 **우회 비용을 계속 끌어올리고 새 공격을 데이터로 학습**하는 자기진화 구조다.